ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Name: Vaishnavi S

Register number :192521313

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/vaish63804/C--192521313--Vaishnavi.S

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

• Write a default constructor, a parameterised constructor and overloaded constructors for the three situations

• Use a member initializer list in every constructor

• Use constructor delegation to avoid repeating code

• Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

• Explain when the compiler supplies a default constructor and when it does not

• Explain why member initializer lists are preferred to assignment inside the constructor body

• Explain the order in which members are initialised (declaration order, not list order) • Draw the state of three objects after construction

Experimental Investigation

You must:

• Create Book objects with every constructor and print them

• Remove the default constructor and record the compiler error when an array of Books is created

• Demonstrate a member initialised in the wrong order and the -Wreorder warning • Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

Code:

```
#include <iostream>

using namespace std;


void change(int a, int& b, int* c) {

    a += 10;

    b += 10;

    *c += 10;

}


int& bigger(int& p, int& q) {

    return p > q ? p : q;

}


void show(int x, int y = 5, int z = 10) {

    cout << x + y + z << " ";

}


void applyByValue(long long fee, int percent) {

    fee -= fee * percent / 100;

}


void applyConcession(long long& fee, int percent) {
```

```
    fee -= fee * percent / 100;
}

void applyByPointer(long long* fee, int percent) {
    if (fee != nullptr) {
        *fee -= *fee * percent / 100;
    }
}

int main() {
    int a = 1, b = 2, c = 3;

    change(a, b, &c);
    cout << a << " " << b << " " << c << '\n';

    bigger(a, c) = 100;
    cout << a << " " << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    cout << '\n';
```

```
    long long hostelFee = 45000;


    applyByValue(hostelFee, 10);
    cout << "after call by value: " << hostelFee << '\n';


    applyConcession(hostelFee, 10);
    cout << "after call by reference: " << hostelFee << '\n';


    long long sportsFee = 45000;
    applyByPointer(&sportsFee, 15);
    cout << "after call by pointer: " << sportsFee << '\n';


    return 0;
}
```

PROGRAM EDITOR

C++ Run Save

```cpp
#include <iostream>
using namespace std;

void change(int a, int& b, int* c) {
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) {
    return p > q ? p : q;
}

void show(int x, int y = 5, int z = 10) {
    cout << x + y + z << " ";
}

void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
```

OUTPUT

```
1 12 13
1 100
16 13 6
after call by value: 45000
after call by reference: 40500
after call by pointer: 38250
```

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

• Implement the class first without a copy constructor and observe the problem

• Implement a deep-copying copy constructor, a copy assignment operator and a destructor

• Verify that copies are independent

• Run the program under -fsanitize=address (or valgrind) for both versions

Design & Analysis

You must:

• Draw memory diagrams for a shallow copy and a deep copy

• Explain why the shallow version crashes at the end (double delete) • State the Rule of Three and explain why the three functions go together • Explain the three situations in which the copy constructor is called

Experimental Investigation

You must:

• Add a message in the copy constructor and count how many copies are made when a Prescription is passed by value and by const reference

• Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines • Report the sanitizer output for both versions

• Test self-assignment (p = p)

Code:

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int size) {
    int highest = 0;

    for (int i = 1; i < size; i++) {
        if (marks[i] > marks[highest]) {
            highest = i;
        }
    }

    return marks[highest];
}

// Returning a reference to a local variable is unsafe.
// Do not call this function.
int& dangerousFunction() {
    int value = 100;
    return value; // Dangling reference: undefined behaviour
}

int highestIndex(const int marks[], int size) {
    int highest = 0;

    for (int i = 1; i < size; i++) {
        if (marks[i] > marks[highest]) {
            highest = i;
        }
    }
```

```
    return highest;
}

int* highestPointer(int marks[], int size) {
    int index = highestIndex(marks, size);
    return &marks[index];
}

int main() {
    int marks[] = {75, 88, 92, 81, 90};
    int size = 5;

    cout << "Original highest mark: "
         << highestMark(marks, size) << endl;

    highestMark(marks, size) += 2;

    cout << "After moderation: "
         << highestMark(marks, size) << endl;

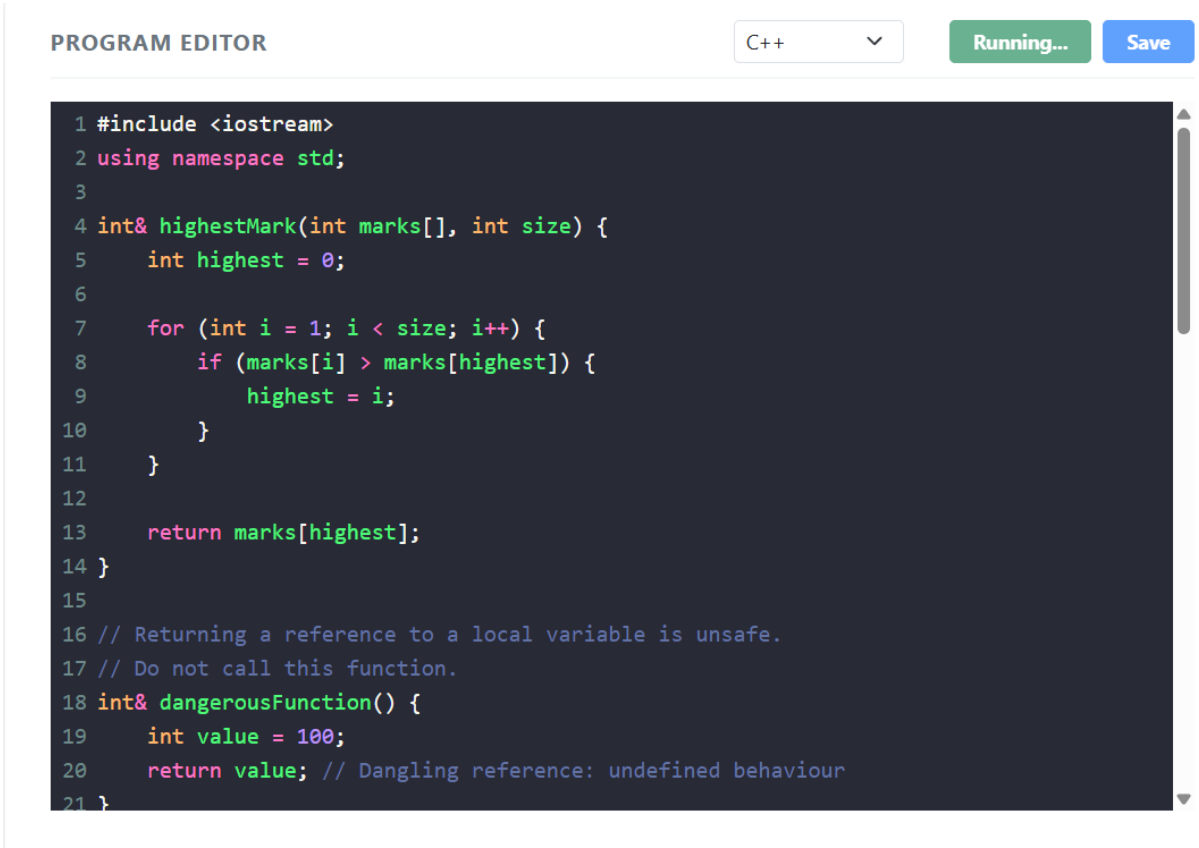
    highestMark(marks, size) = 100;

    cout << "After direct assignment: "
         << highestMark(marks, size) << endl;

    int index = highestIndex(marks, size);
    cout << "Highest mark using index: "
         << marks[index] << endl;
```

```
    int* ptr = highestPointer(marks, size);
    cout << "Highest mark using pointer: "
         << *ptr << endl;

    return 0;
}
```



OUTPUT

```
Original highest mark: 92
After moderation: 94
After direct assignment: 100
Highest mark using index: 100
Highest mark using pointer: 100
```

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

• Write a dynamic constructor that allocates a two-dimensional seat map with new • Write a destructor that releases every allocation and prints a summary • Create events on the stack, on the heap (new/delete) and inside a block • Book and cancel seats

Design & Analysis

You must:

• Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects
• Explain what happens if delete is forgotten, and if delete is used instead of delete[] • Explain why the destructor takes no arguments and cannot be overloaded • Draw the

memory layout of the seat map

Experimental Investigation

You must:

• Print messages in the constructor and destructor and record the order for each kind of object

• Run with -fsanitize=address, first with a missing delete and then with the correct code • Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps • Test an event with zero rows

Code:

```
#include <iostream>

using namespace std;


struct Parcel {

   double weight;

   bool express;

   bool insured;

};


double charge(double weight,

              bool express = false,

              bool insured = false) {

   double amount = weight * 50;


   if (express)
```

```
        amount += 100;

    if (insured)
        amount += 50;

    return amount;
}

// Weight in grams
double charge(int grams) {
    return (grams / 1000.0) * 50;
}

// Parcel object
double charge(const Parcel& p) {
    return charge(p.weight, p.express, p.insured);
}

int main() {
    cout << "charge(2): "
         << charge(2) << endl;

    cout << "charge(2.0): "
         << charge(2.0) << endl;
```

```
    cout << "charge(2.0f): "
         << charge(2.0f) << endl;

    cout << "charge(2000): "
         << charge(2000) << endl;

    cout << "charge(2, true): "
         << charge(2.0, true) << endl;

    Parcel p = {2.0, true, true};

    cout << "charge(Parcel): "
         << charge(p) << endl;

    return 0;
}
```

PROGRAM EDITOR C++ Run Save

```cpp
#include <iostream>
using namespace std;

struct Parcel {
    double weight;
    bool express;
    bool insured;
};

double charge(double weight,
              bool express = false,
              bool insured = false) {
    double amount = weight * 50;

    if (express)
        amount += 100;

    if (insured)
        amount += 50;

    return amount;
```

OUTPUT

```
charge(2): 0.1
charge(2.0): 100
charge(2.0f): 100
charge(2000): 100
charge(2, true): 200
charge(Parcel): 250
```

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise.

The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

Implementation Tasks

You must:

• Write a Money class storing paise in a long long

• Overload +, -, +=, == and < as appropriate

• Overload << (and >>) for printing and reading amounts

• Write the bill calculation using the overloaded operators

Design & Analysis

You must:

• Explain why + returns a new object while += returns a reference to *this • Decide which operators are members and which are friends, and justify each • Explain why << must be a non-member function

• Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

• Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50) • Test subtraction that produces a negative amount and decide how to handle it • Compare the readability of the bill code with and without operator overloading • Compare the result with a double-based version for 1,000,000 additions

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    int start;

    void addTransaction(double amount) {
        int position;

        if (count < 10) {
            position = (start + count) % 10;
            count++;
        } else {
            position = start;
            start = (start + 1) % 10;
        }

        transactions[position] = amount;
    }

public:
    SavingsAccount(long long accNo, double initialBalance)
        : accountNumber(accNo),
          balance(initialBalance),
          count(0),
          start(0) {
    }

    void deposit(double amount) {
        if (amount <= 0) {
            cout << "Invalid deposit" << endl;
```

```
            return;
        }

        balance += amount;
        addTransaction(amount);
        cout << "Deposited: Rs " << amount << endl;
    }

    void withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            cout << "Invalid withdrawal" << endl;
            return;
        }

        balance -= amount;
        addTransaction(-amount);
        cout << "Withdrawn: Rs " << amount << endl;
    }

    void miniStatement() const;

    void showBalance() const {
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: Rs " << balance << endl;
    }
};

void SavingsAccount::miniStatement() const {
    cout << "\nLast Transactions\n";

    for (int i = 0; i < count; i++) {
        int position = (start + i) % 10;
```

```
        if (transactions[position] >= 0) {
            cout << "Deposit: Rs "
                 << transactions[position] << endl;
        } else {
            cout << "Withdrawal: Rs "
                 << -transactions[position] << endl;
        }
    }
}

int main() {
    SavingsAccount account(123456, 5000);

    account.deposit(1000);
    account.withdraw(500);
    account.deposit(2000);
    account.withdraw(300);

    account.showBalance();
    account.miniStatement();

    return 0;
}
```

PROGRAM EDITOR C++ Run Save

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    int start;

    void addTransaction(double amount) {
        int position;

        if (count < 10) {
            position = (start + count) % 10;
            count++;
        } else {
            position = start;
            start = (start + 1) % 10;
```

OUTPUT

```
Deposited: Rs 1000
Withdrawn: Rs 500
Deposited: Rs 2000
Withdrawn: Rs 300
Account Number: 123456
Balance: Rs 7200

Last Transactions
Deposit: Rs 1000
Withdrawal: Rs 500
```

Q5 – Overloading Increment, Comparison and Subscript Operators 6

Marks Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

• Overload prefix and postfix ++ for Time

• Overload == and < for Time

• Overload [] for Playlist with a range check, in both a const and a non-const version • Use the operators in a short playback simulation

Design & Analysis

You must:

• Explain how the compiler tells prefix from postfix ++ (the int dummy parameter) • Explain why postfix ++ must return a copy and is slower

• Explain why operator[] needs a const and a non-const version

• Explain what should happen when an index is out of range

Experimental Investigation

You must:

• Verify the values of a and b after a = t++ and b = ++t

• Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2 • Test playlist[-1] and playlist[size]

• Test minute and hour roll-over (59:59 → 1:00:00)

Code:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class BusTicket {
private:
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    string name;

public:
    explicit BusTicket(const string& n)
        : seat(0), name(n) {

        if (issued < CAPACITY) {
            issued++;
            seat = issued;
        }
    }

    static int count() {
        return issued;
    }

    void print() const {
        cout << name << ": ";

        if (seat != 0)
            cout << "Seat " << seat;
        else
            cout << "Waiting";
```

```
        cout << '\n';
    }
};

int BusTicket::issued = 0;

void createTickets() {
    BusTicket tickets[] = {
        BusTicket("Asha"),
        BusTicket("Bala")
    };

    for (const BusTicket& t : tickets) {
        t.print();
    }

    cout << "Total issued: " << BusTicket::count() << endl;
}

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip) {
        t.print();
    }
```

```
    cout << "Issued: " << BusTicket::count() << endl;

    cout << "\nCreating more tickets:\n";
    createTickets();

    return 0;
}
```

PROGRAM EDITOR C++ Running... Save

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
private:
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    string name;

public:
    explicit BusTicket(const string& n)
        : seat(0), name(n) {

        if (issued < CAPACITY) {
            issued++;
            seat = issued;
        }
    }
```

OUTPUT

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

Creating more tickets:
Asha: Waiting
Bala: Waiting
Total issued: 3
```

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

Implementation Tasks

You must:

• Implement Text with a constructor, copy constructor, copy assignment and destructor • Overload + for concatenation, == and != for comparison and << for printing • Prevent self-assignment problems

• Test long messages and empty messages

Design & Analysis

You must:

• List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why

• Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type

• Explain why operator= must be a member function

• Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

• Count the copies made when a + b + c is evaluated

• Run the program under the address sanitizer and fix any reported error • Compare your Text class with std::string for 100,000 concatenations • Test assigning a Text to itself

Code:

```
#include <iostream>

using namespace std;


class Time {

private:

    int h, m;


public:

    Time(int hh, int mm) : h(hh), m(mm) {}


    friend inline int minutesBetween(

        const Time& a, const Time& b
```

```
    );

    friend inline bool earlier(
        const Time& a, const Time& b
    );
};

inline int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;

    return (first > second)
        ? first - second
        : second - first;
}

inline bool earlier(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;

    return first < second;
}
```

```
int main() {
    Time departure(9, 45);
    Time arrival(13, 10);

    cout << minutesBetween(departure, arrival) << " "
        << boolalpha
        << earlier(arrival, departure) << endl;

    Time morning(11, 30);
    Time afternoon(12, 30);

    cout << "Minutes between times: "
        << minutesBetween(morning, afternoon) << endl;

    cout << "Morning is earlier: "
        << earlier(morning, afternoon) << endl;

    Time same1(10, 30);
    Time same2(10, 30);

    cout << "Same time difference: "
        << minutesBetween(same1, same2) << endl;
```

```
    cout << "Same time comparison: "
         << earlier(same1, same2) << endl;

    return 0;
}
```

PROGRAM EDITOR

C++

Running...

Save

```
#include <iostream>
using namespace std;

class Time {
private:
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    friend inline int minutesBetween(
        const Time& a, const Time& b
    );

    friend inline bool earlier(
        const Time& a, const Time& b
    );
};

inline int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
```

OUTPUT

```
205 false
Minutes between times: 60
Morning is earlier: true
Same time difference: 0
Same time comparison: false
```

Q7 – Design a Complex-Number Class from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

• Identify the data and the operations needed

• Identify which constructors are needed (default, from real part, from real and imaginary parts)

• State the requirements as measurable statements

Stage 2 – Design | 3 Marks

• Design the Complex class with constructors and overloaded +, -, *, /, == and << • Decide which operators are members and which are friends

• Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

• Implement the class and compute the impedance of a series and a parallel RLC circuit

• Verify the results by hand calculation

• Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

• Compare your class with std::complex<double>

• Explain why each constructor and operator is needed

• Identify one remaining limitation

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Code:

```
#include <iostream>

using namespace std;


class SavingsAccount {

private:

    long long accountNumber;

    double balance;

    double transactions[10];

    int count;

    int start;


public:

    SavingsAccount(long long accNo, double amount) {

        accountNumber = accNo;
```

```
    balance = amount;
    count = 0;
    start = 0;
}

void addTransaction(double amount) {
    int position;

    if (count < 10) {
        position = (start + count) % 10;
        count++;
    } else {
        position = start;
        start = (start + 1) % 10;
    }

    transactions[position] = amount;
}

void deposit(double amount) {
    if (amount <= 0) {
        cout << "Invalid deposit" << endl;
        return;
```

```
    }

    balance += amount;
    addTransaction(amount);
}

void withdraw(double amount) {
    if (amount <= 0 || amount > balance) {
        cout << "Invalid withdrawal" << endl;
        return;
    }

    balance -= amount;
    addTransaction(-amount);
}

void miniStatement() const {
    cout << "\nAccount Number: "
         << accountNumber << endl;

    cout << "Current Balance: Rs "
         << balance << endl;
```

```
        cout << "\nLast 10 Transactions:\n";

        for (int i = 0; i < count; i++) {
            int position = (start + i) % 10;

            if (transactions[position] >= 0) {
                cout << "Deposit: Rs "
                     << transactions[position] << endl;
            } else {
                cout << "Withdrawal: Rs "
                     << -transactions[position] << endl;
            }
        }
    }
};

int main() {
    SavingsAccount account(123456, 10000);

    for (int i = 1; i <= 12; i++) {
        account.deposit(100 * i);
    }
```

```
    account.miniStatement();


    return 0;
}
```

PROGRAM EDITOR

C++ | Run | Save

```
#include <iostream>
using namespace std;

class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    int start;

public:
    SavingsAccount(long long accNo, double amount) {
        accountNumber = accNo;
        balance = amount;
        count = 0;
        start = 0;
    }

    void addTransaction(double amount) {
        int position;
```

OUTPUT

```
Account Number: 123456
Current Balance: Rs 17800

Last 10 Transactions:
Deposit: Rs 300
Deposit: Rs 400
Deposit: Rs 500
Deposit: Rs 600
Deposit: Rs 700
Deposit: Rs 800
```

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your report containing the required evidence for Q1–Q7

• C++ source code for all questions

• Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.